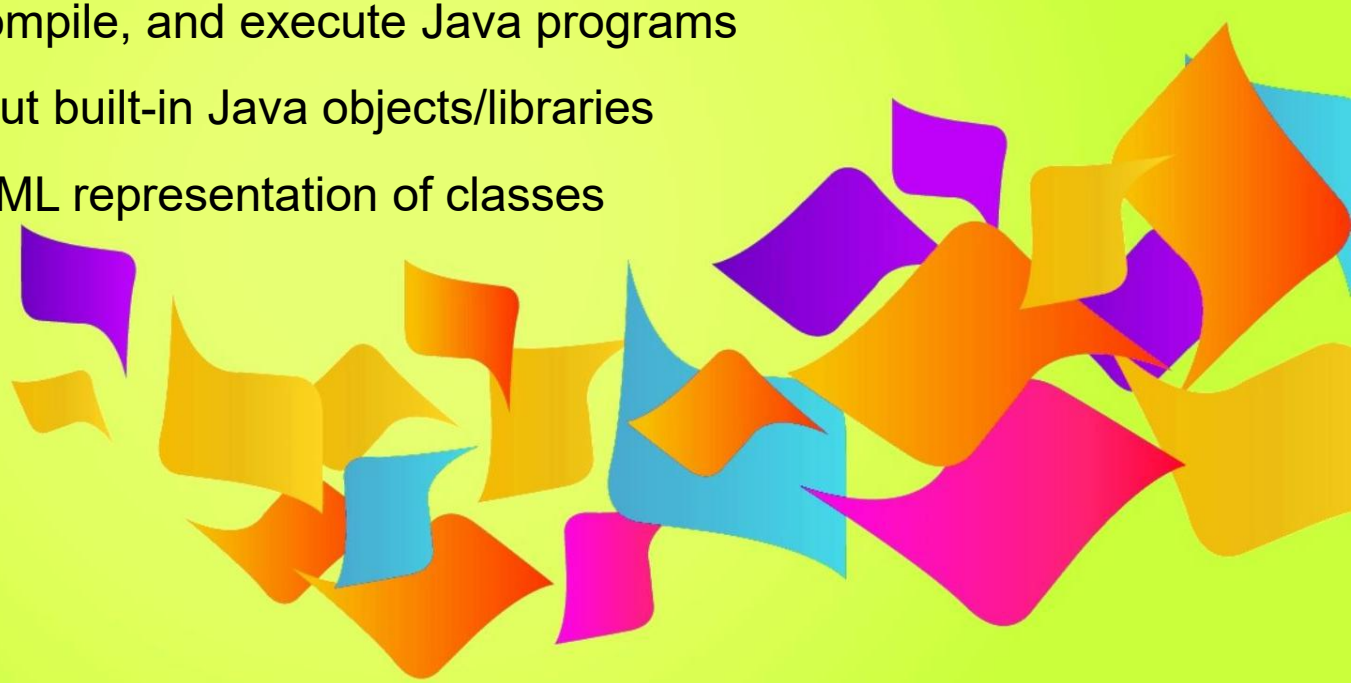
Support Material for Lecture 02

OOP with Java & UML – Part 1: Classes & Structure

- Understand Java programming basics
- Know the syntax of Java components
- Create, compile, and execute Java programs
- Learn about built-in Java objects/libraries
- Explore UML representation of classes



What is Object-Oriented Programming (OOP)?

A Programming paradigm based on **objects** that combine **data and behavior**

Focuses on **real-world modeling**

Main OOP features:

Encapsulation
Inheritance
Polymorphism
Abstraction

Example: A “Car” object with attributes (color, model) and methods (start(), stop()).

Classes and Objects

Class → Blueprint or template for objects
Object → Instance of a class

✓ Java allows creation of many objects from one class

Car myCar = new Car();

Concept	Example
Class	Blueprint of a car
Object	Actual car built from the blueprint
Attributes	Color, model, year
Methods	Start(), accelerate(), stop()

```
// File name: Car.java
public class Car {

    // Attributes (fields)
    String brand;
    String color;
    int year;

    // Constructor (runs when an object is created)
    public Car() {
        brand = "Toyota";
        color = "Red";
        year = 2024;
    }

    // Method (behavior)
    void displayDetails() {
        System.out.println("Brand: " + brand);
        System.out.println("Color: " + color);
        System.out.println("Year: " + year);
    }

    // Main method (program entry point)
    public static void main(String[] args) {
        // Object creation
        Car myCar = new Car();

        // Accessing object's method
        myCar.displayDetails();
    }
}
```

Key Java Rules

✅ Class name = File name

✅ One public class per file

✅ Main method required for execution

❌ Class cannot have parentheses ()

Activity: Identify Class Components

```
public class Student
{
int id;
String name;
void displayInfo() {
System.out.println(id + " " + name);
}
}
```

Questions:

Which part is the **class name**?

Which are the **attributes**?

Which is the **method**?

Why Main Method is Important

Without main(), the JVM has **no entry point** to execute the program.

We need to add following code with previous one to run it.

```
public static void main(String[] args)
{
    Student std = new Student();
    std.id = 1;
    std.name = "Ali";
    std.displayInfo();
}
```

Understanding Class Modifiers

- **A class modifier in Java defines how accessible a class is ?**

i.e., where it can be used in a program.

Modifier	Access	Example
public	Any package	public class Account
(default)	Same package only	class MyProg

Access Specifiers Simplified

Specifier	Visibility	Use For
public	Everywhere	**Methods shared across classes**
private	Inside class only	**Data hiding**
protected	Same package + subclasses	**Inheritance**
(default)	Same package	**Helper classes**

Data Type Conversion

- In Java, **data type conversion** happens when you change a value from one data type to another.

There are **two types**:

- **Widening Conversion (Safe)**
  Converts smaller type → larger type automatically (no data loss).
  Example: int → double

```
int a = 10;

double b = a;     // int → double (safe)                    output  10.0

System.out.println(b);
```

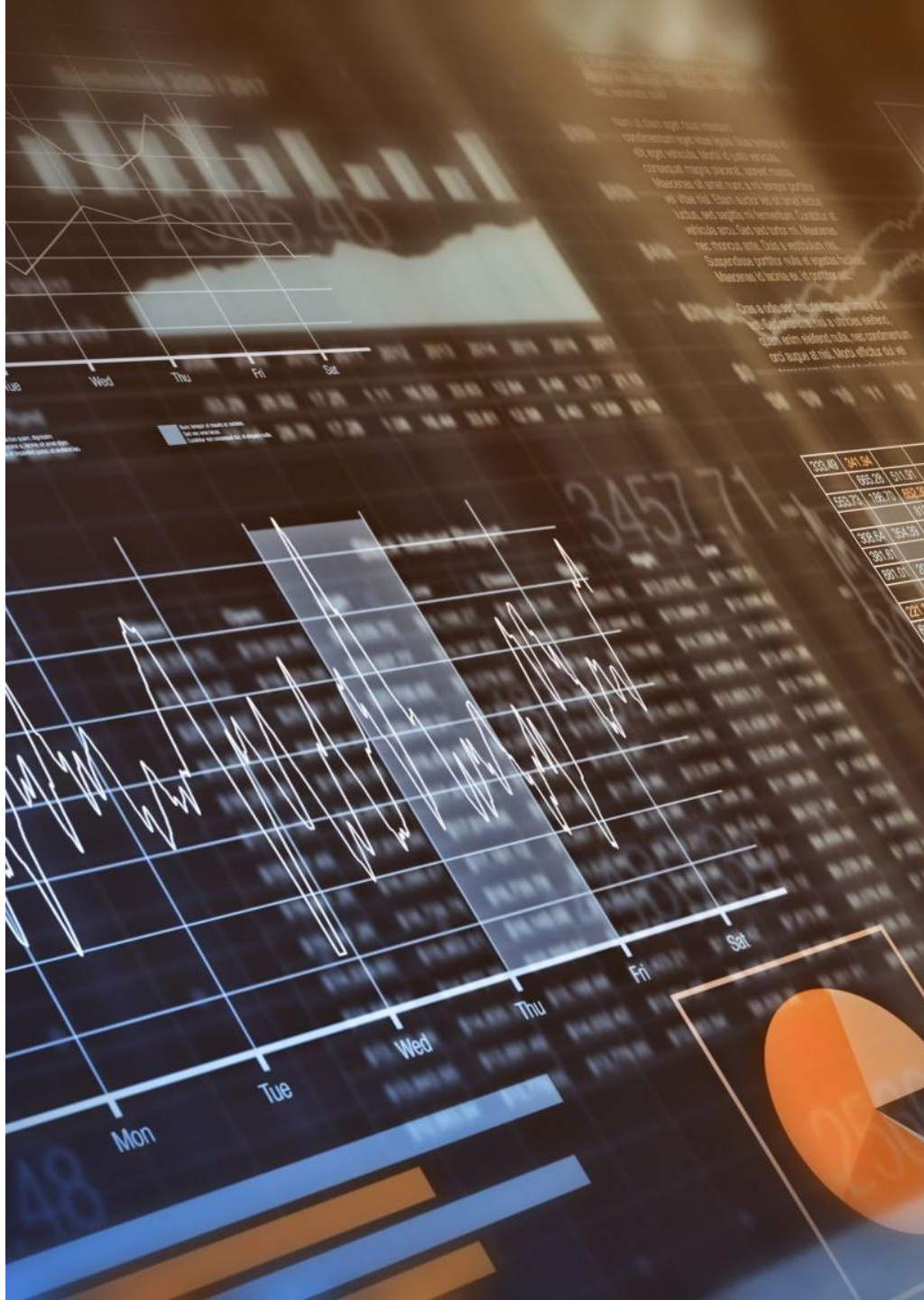
- **Narrowing Conversion (Risky)**
  Converts larger type → smaller type manually (possible data loss).
  Example: double → int

```
double x = 9.75;

int y = (int) x;   // double → int (possible data loss)

 System.out.println(y);
```

Output : 9



Data Type Conversion (Contd.)

There are **three ways** to convert:

- **Assignment conversion** – automatic widening when assigning.

```
int a = 10;
double b = a;     // int → double (safe)
```

- **Promotion** – automatic widening inside an expression.

```
int n1 = 5;
double n2 = 2.5;
double result = n1 + n2; // int promoted to double
System.out.println(result);
```

- **Casting** – manual conversion using (type) syntax. (Casting is when a programmer **explicitly converts** one data type into another by writing the **target type in parentheses** before the value or variable.)

```
double price = 99.75;
int roundedPrice = (int) price;  // Manual cast
```